

```
# import python data science libraries and understand why each one exists
#NumPy - Numerical operations
#Pandas - Data handling & analysis
#Matplotlib - Basic data visualization
#Seaborn - Advanced data visualization
```

```
# The Dataset
# Request id-Unique ID for each ride request
# Pickup point-Where the rider is: 'City' or 'Airport'
# Driver id-The driver assigned (or attempted)
#Status-Trip Completed / Cancelled / No Cars Available
#Request timestamp-Date & time the rider pressed 'Book Ride'
#Drop timestamp-Date & time of drop-off (empty if cancelled)''
```

```
# Our Toolkit (The Kitchen Analogy)
# Python = Kitchen with only a stove and sink
# NumPy = A fast scientific calculator
# Pandas = An Excel spreadsheet (but 100x more powerful)
# Matplotlib = Excel's "Insert Chart" button
# Seaborn = A smarter, prettier chart tool
```

```
# import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
#libraries-->shortcuts
#numpy - np
#pandas - pd
#Matplotlib - plt
#seaborn- sns
```

```
sns.set_style("whitegrid")
```

Numpy-Numerical python

```
df.shape
```

```
(6745, 9)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6745 entries, 0 to 6744
Data columns (total 6 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Request id            6745 non-null   int64
1   Pickup point          6745 non-null   object
2   Driver id             4095 non-null   float64
3   Status                6745 non-null   object
4   Request timestamp     6745 non-null   object
5   Drop timestamp        2831 non-null   object
dtypes: float64(1), int64(1), object(4)
memory usage: 316.3+ KB
```

```
df.describe()
```

	Request id	Driver id	Request timestamp	Drop timestamp
count	6745.000000	4095.000000	6745	2831
mean	3384.644922	149.501343	2016-09-04 08:49:18.446849536	2016-09-05 17:03:26.523843072
min	1.000000	1.000000	2016-07-13 00:00:24	2016-07-13 00:01:12
25%	1691.000000	75.000000	2016-07-14 07:38:26	2016-07-14 08:10:24.500000
50%	3387.000000	149.000000	2016-07-15 13:44:15	2016-07-15 15:27:41
75%	5080.000000	224.000000	2016-11-07 19:00:00	2016-11-07 19:25:30
max	6766.000000	300.000000	2016-12-07 23:54:00	2016-12-07 23:45:00
std	1955.099667	86.051994	NaN	NaN

```
df['Request timestamp']=pd.to_datetime(df['Request timestamp'])
df['Drop timestamp']=pd.to_datetime(df['Drop timestamp'])
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6745 entries, 0 to 6744
Data columns (total 6 columns):
#   Column              Non-Null Count  Dtype
---  -
0   Request id          6745 non-null   int64
1   Pickup point        6745 non-null   object
2   Driver id           4095 non-null   float64
3   Status              6745 non-null   object
4   Request timestamp   6745 non-null   datetime64[ns]
5   Drop timestamp      2831 non-null   datetime64[ns]
dtypes: datetime64[ns](2), float64(1), int64(1), object(2)
memory usage: 316.3+ KB
```

```
df['Requesthour']=df['Request timestamp'].dt.hour
df.head() # creating new column
```

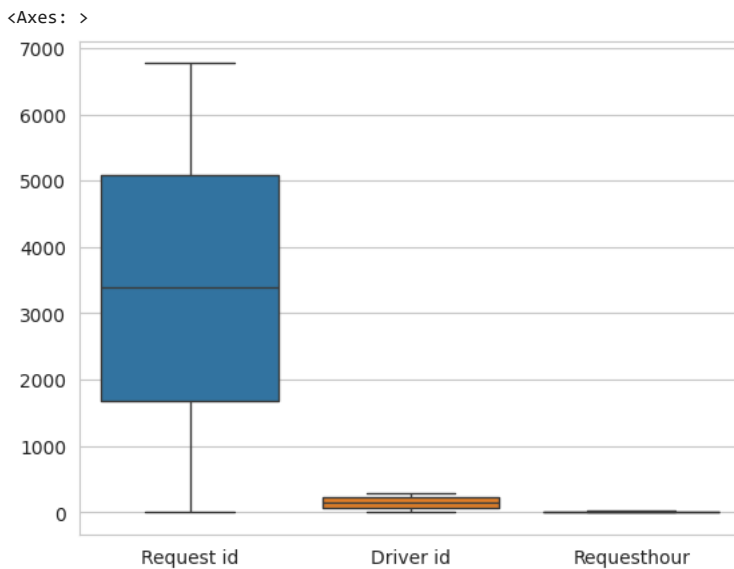
	Request id	Pickup point	Driver id	Status	Request timestamp	Drop timestamp	Requesthour
0	619	Airport	1.0	Trip Completed	2016-11-07 11:51:00	2016-11-07 13:00:00	11
1	867	Airport	1.0	Trip Completed	2016-11-07 17:57:00	2016-11-07 18:47:00	17
2	1807	City	1.0	Trip Completed	2016-12-07 09:17:00	2016-12-07 09:58:00	9
3	2532	Airport	1.0	Trip Completed	2016-12-07 21:08:00	2016-12-07 22:03:00	21
4	3112	City	1.0	Trip Completed	2016-07-13 08:33:16	2016-07-13 09:25:47	8

```
df.isnull().sum()
```

	0
Request id	0
Pickup point	0
Driver id	2650
Status	0
Request timestamp	0
Drop timestamp	3914
Requesthour	0
TimeSlot	0
Cab Availability	0

dtype: int64

```
sns.boxplot(df)
```



[Show code](#)

[Show hidden output](#)

```
df['Status'].value_counts()
```

	count
Trip Completed	2831
No Cars Available	2650
Cancelled	1264

dtype: int64

```
def assign_timeslot(hour):
    if hour <= 4:    return 'Dawn'
    elif hour <= 9:  return 'Early Morning'
    elif hour <= 16: return 'Noon'
    elif hour <= 21: return 'Late Evening'
    else:            return 'Night'
```

```
df['TimeSlot']=df['Requesthour'].apply(assign_timeslot)
df.head()
```

	Request id	Pickup point	Driver id	Status	Request timestamp	Drop timestamp	Requesthour	TimeSlot
0	619	Airport	1.0	Trip Completed	2016-11-07 11:51:00	2016-11-07 13:00:00	11	Noon
1	867	Airport	1.0	Trip Completed	2016-11-07 17:57:00	2016-11-07 18:47:00	17	Late Evening
2	1807	City	1.0	Trip Completed	2016-12-07 09:17:00	2016-12-07 09:58:00	9	Early Morning
3	2532	Airport	1.0	Trip Completed	2016-12-07 21:08:00	2016-12-07 22:03:00	21	Late Evening
4	3112	City	1.0	Trip Completed	2016-07-13 08:33:16	2016-07-13 09:25:47	8	Early Morning

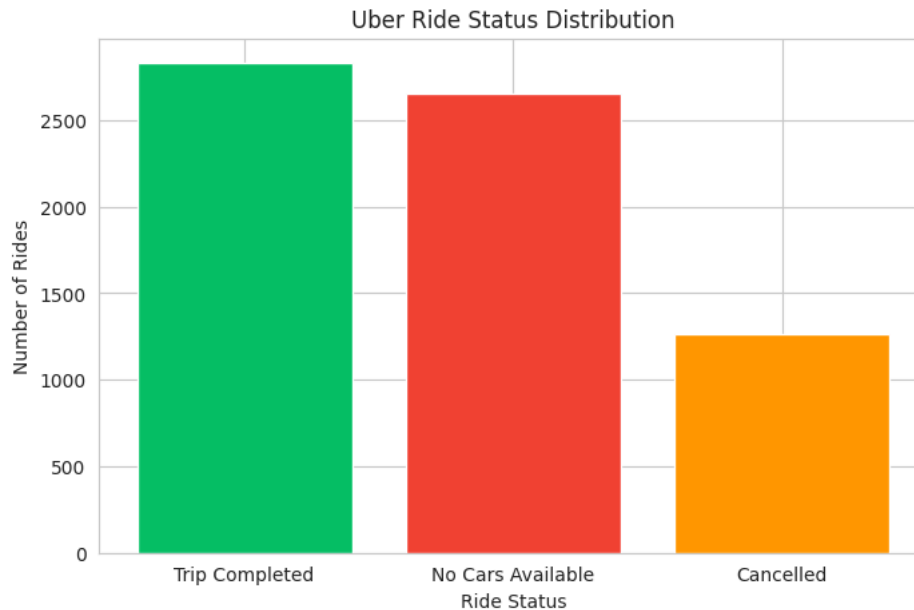
```
df['Cab Availability']=df['Status'].apply(lambda x: 'Available' if x=='Trip Completed' else 'not Available')
```

```
df['Cab Availability'].value_counts(normalize=True)
```

proportion	
Cab Availability	
not Available	0.580282
Available	0.419718

dtype: float64

```
Status_counts = df['Status'].value_counts()
plt.figure(figsize=(8,5))
plt.bar(Status_counts.index, Status_counts.values, color=['#06C167', '#F44336', '#FF9800'])
plt.title('Uber Ride Status Distribution')
plt.xlabel('Ride Status')
plt.ylabel('Number of Rides')
plt.show()
```

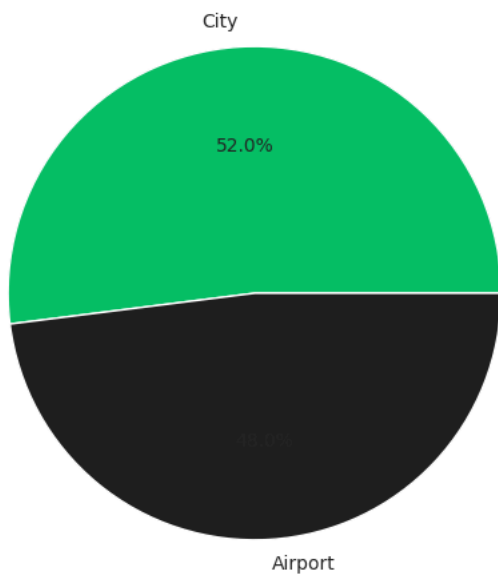


Status_counts.index

Index(['Trip Completed', 'No Cars Available', 'Cancelled'], dtype='object', name='Status')

```
pickup_counts = df['Pickup point'].value_counts()
plt.figure(figsize=(6, 6))
plt.pie(pickup_counts.values,
labels=pickup_counts.index,
autopct='%1.1f%%',
colors=['#06C167', '#222222'])
plt.title('Ride Requests: City vs Airport')
plt.show()
print(" Almost equal split both routes have problems.")
```

Ride Requests: City vs Airport



Almost equal split both routes have problems.

```
df ['Pickup point'].value_counts()
```

	count
Pickup point	
City	3507
Airport	3238

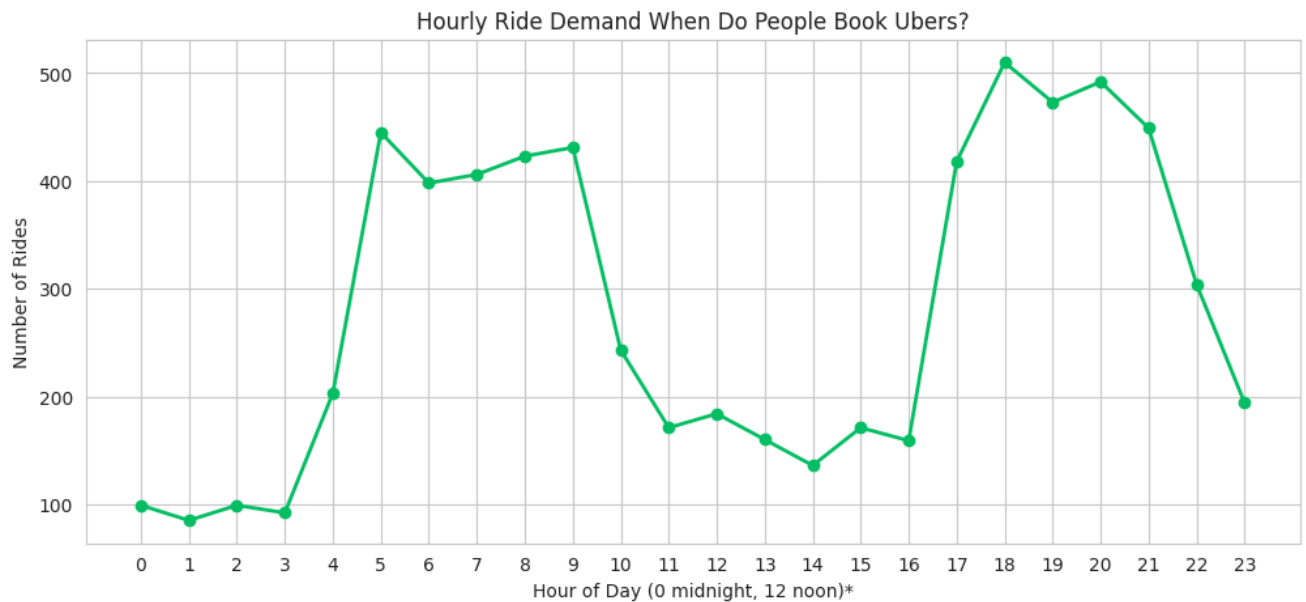
dtype: int64

```
df.groupby('Requesthour').size()
```

	0
Requesthour	
0	99
1	85
2	99
3	92
4	203
5	445
6	398
7	406
8	423
9	431
10	243
11	171
12	184
13	160
14	136
15	171
16	159
17	418
18	510
19	473
20	492
21	449
22	304
23	194

dtype: int64

```
hourly_demand = df.groupby('Requesthour').size()
#groupby('Requesthour') groups rows by hour
#.size() counts the rows in each group
plt.figure(figsize=(12, 5))
plt.plot(hourly_demand.index, hourly_demand.values,
color='#06C167', linewidth=2, marker='o')
plt.title('Hourly Ride Demand When Do People Book Ubers?')
plt.xlabel('Hour of Day (0 midnight, 12 noon)*')
plt.ylabel('Number of Rides')
plt.xticks(range(0, 24))
plt.show()
```



```
plt.figure(figsize=(10,6))
sns.countplot(data=df,
              x='Pickup point',
              hue='Status',
              palette=['#F44336', '#FF9800', '#06C167'])
plt.title('Ride Outcomes: City vs Airport')
plt.xlabel('Pickup Location')
plt.ylabel('Number of Rides')
plt.show()
print("HIGH Cancellations Airport, HIGH No Cars Available") # Corrected to a valid print statement
print("Two different problems, two different solutions needed!") # Corrected to a valid print statement
```



HIGH Cancellations Airport, HIGH No Cars Available
 Two different problems, two different solutions needed!

```
df.groupby(['Requesthour', 'Status']).size().unstack(fill_value=0)
```

	Status	Cancelled	No Cars Available	Trip Completed
Requesthour				
0		3	56	40
1		4	56	25
2		5	57	37
3		2	56	34
4		51	74	78
5		176	84	185
6		145	86	167
7		169	63	174
8		178	90	155
9		175	83	173
10		62	65	116
11		15	41	115
12		19	44	121
13		18	53	89
14		11	37	88
15		21	48	102
16		22	46	91
17		35	232	151
18		24	322	164
19		24	283	166
20		41	290	161
21		42	265	142
22		12	138	154
23		10	81	103

```
# This line is causing a SyntaxError due to incorrect syntax.
# It seems you were trying to filter the DataFrame for 'City' pickup points.
# The correct way to do this is:
df[df['Pickup point'] == 'City']
```

	Request id	Pickup point	Driver id	Status	Request timestamp	Drop timestamp	Requesthour	TimeSlot	Cab Availability
2	1807	City	1.0	Trip Completed	2016-12-07 09:17:00	2016-12-07 09:58:00	9	Early Morning	Available
4	3112	City	1.0	Trip Completed	2016-07-13 08:33:16	2016-07-13 09:25:47	8	Early Morning	Available
8	6248	City	1.0	Trip Completed	2016-07-15 17:57:27	2016-07-15 18:50:51	17	Late Evening	Available
9	267	City	2.0	Trip Completed	2016-11-07 06:46:00	2016-11-07 07:25:00	6	Early Morning	Available
11	1983	City	2.0	Trip Completed	2016-12-07 12:30:00	2016-12-07 12:57:00	12	Noon	Available
...	...	...	...	...	...	...	...	...	...
6738	6746	City	NaN	No Cars Available	2016-07-15 23:46:03	NaT	23	Night	not Available
6739	6739	City	NaN	No Cars Available	2016-07-15 23:46:00	NaT	23	Night	not Available

```
hourly_status_df = df.groupby(['Requesthour', 'Status']).size().unstack(fill_value=0)
hourly_status_df.plot(kind='bar', stacked=True, figsize=(14,6),
color=['#F44336', '#FF9800', '#06C167'])
```

